

assignment 3 - 530577265

Algorithm

We encode the three-stage supply chain as a single-commodity circulation with explicit lower and upper bounds on some edges. Two special vertices, s and t , mark the formal beginning and end of every starfighter's journey. For each system $j \in \{1, \dots, m\}$ we introduce three "stage hubs": F_j for the frame stage, E_j for the engines stage, and O_j for the fitout stage. Every factory $u \in \{1, \dots, n\}$ is represented by a pair of vertices u_{in} and u_{out} connected by a single internal edge. Giving that internal edge lower bound a_u and capacity b_u forces the factory to produce between those limits; all other edges incident on the factory remain unbounded except by the relevant quotas. Raw frames (before factory processing) enter the network through edges (s, F_j) of infinite capacity (from a supersource). From F_j flow can reach any frame factory in that system and once frames are produced they travel directly to the engines hub of whatever system will handle the next stage (E_j). If that destination differs from the source system, the edge is capped by the corresponding frame-quota q_{frame_ij} ; intra-system links are left unbounded. The engines factory processing method is wired in the same fashion: E_j feeds engines factories, their internal edges enforce a_u / b_u , and their outputs move to fitout hubs with capacities equal to the engine-quota matrix. A fitout factory's output goes straight to t (a supersink). Finally we insert one edge (t, s) whose lower and upper bounds are both X , requiring the circulation to complete exactly X fighters.

Lower-bound elimination ($c'(e) = c(e) - l(e)$) converts the bounded-circulation instance into an ordinary $s'-t'$ maximum-flow instance: every lower bound is subtracted from its edge, vertex surpluses are balanced by edges from a super-source s' , deficits by edges into a super-sink t' , and the total surplus is denoted D .

We solve that $s'-t'$ instance with **capacity-scaling Ford-Fulkerson** (to clarify: on the residual network of G after lower-bound elim). If the algorithm finds a flow of value D we add the subtracted lower bounds back, read the resulting integral circulation as a production plan, and determine, thus, that it is indeed possible to assemble the supply chain schedule.

Correctness

RTP: A feasible production schedule exists $\iff$ the maximum s' - t' flow in G' (graph after lower bound elimination) has value D .

Assume first that a feasible production schedule exists. Trace one complete path through the graph for each starfighter: s to the frame hub of its initial system, into its chosen frame factory, out to the engines hub of the next system (respecting the appropriate quota edge), through an engines factory, on to the fit-out hub, through a fit-out factory, and finally to t and back along t - s . Because the schedule already respects factory limits, every internal edge carries a value between its bounds; because it respects shipping quotas, every inter-system arc remains within capacity; and because exactly X fighters finish, the arc t - s carries its required value. Thus the schedule induces a feasible circulation and, after lower-bound removal, a feasible s' - t' flow of value D , so the algorithm must accept.

Conversely, suppose the capacity-scaling routine returns value D . Restoring each lower bound yields an integral circulation (since all input numbers are integers and Ford–Fulkerson preserves integrality). Flow conservation and the directed structure of the graph ensure every unit of flow follows the sequence *frame* $\rightarrow$ *engines* $\rightarrow$ *fitout* exactly once. The integral value on each internal edge lies in the interval $[a_u, b_u]$, satisfying every factory's contractual limits; capacities on quota edges ensure no inter-system shipment is excessive; and the fixed edge t - s guarantees that precisely X units reach completion. Interpreting each s - t path as one starfighter therefore produces a valid schedule.

Time complexity⁴

Let n = number of **factories**, m = number of star systems, and $U = \max\{b_u, q_{ij}^{\text{frame}}, q_{ij}^{\text{engine}}, X\}$ (essentially the edge with the largest cap).

The total runtime can be broken down into three parts

1. construction of the flow network,
2. execution of the flow algorithm, and

3. lower-bound pre-processing

1. Construction

The circulation graph has:

- 2 global nodes s & t
- $3m$ system-stage hubs: F_j, E_j, O_j for each system $j = 1, \dots, m$
- $2n$ factory nodes: u_{in}, u_{out} for each factory $u = 1, \dots, n$

So the total number of vertices is $O(n + m)$

The number of edges is determined by:

- n internal edges (one per factory, with bounds $[a_u, b_u]$)
- $2n$ hub-factory arcs (entry and exit wiring)
- Up to $2m^2$ quota arcs across systems (from the q^{frame} and q^{engine} matrices)
- $2m$ auxiliary edges: from s to F_j and from O_j to t
- The fixed loop arc $t-s$

Thus the total number of edges before pre-processing is $O(n + m^2)$.

The lower-bound elimination step adds at most one edge per vertex, so the final graph used for max-flow remains size $O(n + m^2)$

2. Max-flow algorithm

We apply **capacity-scaling Ford-Fulkerson**, which runs in $O(E^2 \log U)$

- here $E = O(n + m^2)$ (total number of edges after transformation).
- The algorithm performs $\log U$ scaling phases (the Δ is set to the smallest power of 2 smaller than U and divide it by 2 every time although F/E could also be used as in the lectures)
- In each phase, it performs at most $O(E)$ augmentations
- Each augmentation is found in $O(E)$ time using DFS on the residual graph

Hence, total max-flow computation takes

$$O((n + m^2)^2 \log U)$$

3. Pre-processing and translation

Constructing the circulation graph and applying the lower-bound elimination both require only $O(E)$ time. Restoring lower bounds after max-flow and reading the resulting circulation as a schedule is also linear in the size of the graph. These steps do not dominate the asymptotic runtime.

Conclusion:

The full algorithm is:

$$O((n + m^2)^2 \log U)$$

where all constants are determined by the number of factories (n), the number of systems (m), and the largest numeric quantity in the input (U).